

sarvam

Assignment

AI Agent Challenge: Deep Research Agent

Duration: 2 days

Stack: Python

Restrictions: Do not use LangChain, LangGraph, CrewAI, LlamaIndex, Haystack or any other framework (build your own agent loop and orchestration using python)

Submission format: Please submit all the required links and necessary assumptions you have made in a single PDF file before the deadline.

Overview

Build a Deep Research Agent that:

- Conducts research directly from the **web** using a search provider (Tavily / Serper / Parallel etc)
- Uses **retrieved web context directly**
- Answers questions with **source citations (URL + title/domain)**
- Maintains **session management + conversation/turn history**
- Streams intermediate progress updates during research

You may use any **free** LLM available online or run locally (open-source models, local inference, free-tier APIs).

Part 1: Design Note (25%)

Write a 1-2 page design note (README section is fine):

Include

- Target users and the problem being solved
- Definition of “deep research” for this implementation
- Success metrics (pick 3-5) you believe best capture research quality
- Data flow and components (search → page fetch → context selection → answer)

- Risks/limitations (rate limits, low-quality sources, conflicting sources, context length limits) and 2 future improvements

Part 2: Technical Implementation (75%)

Web Research

Use **one or more** of: **Tavily**, **Serper**, **Parallel** as your web search layer.

Requirements

- Implement a search module that:
 - issues search queries for a user question
 - returns results with at least: title, url, snippet, optional relevance score
- Implement a page-fetch module that:
 - retrieves full page content for selected URLs
 - extracts readable text (HTML → text)
 - stores metadata: url, title, retrieved_at, optional domain

Context construction

- Select a subset of fetched content to use as LLM context:
 - prioritize relevance, recency, and source diversity
 - limit total context size (tokens or characters)
- Preserve metadata for citation mapping.

Citation requirements

- Every claim-heavy answer must cite sources as:
 - [Title — domain] with URL included, or
 - (domain, URL)
- If sources conflict, the agent must explicitly note the disagreement and cite both.

Session Management and Histories

Persist session state with a session_id (JSON or SQLite).

Conversation history

- Store all user and assistant messages with timestamps

Turn history

For each user query, store:

- query
- search queries issued
- URLs opened
- context snippets selected
- final answer
- timestamp

Context management

Implement a context builder that decides what is sent to the LLM:

- current user query
- rolling summary of prior conversation when history grows long
- most relevant prior turns
- selected web context snippets with metadata

Enforce a maximum context length and summarization fallback.

Agent Flow (Write with Python without using agents framework)

For each user query:

1. Plan

- Produce a short plan describing the research steps and search strategy

2. Search

- Use Tavily / Serper / Parallel to retrieve relevant web results

3. Acquire & Select Context

- Fetch page content for selected URLs
- Select the most relevant snippets to build the LLM context

4. Answer

- Generate a final response grounded in the selected web context

- Include citations using title/domain/URL
- If evidence is weak, missing, or conflicting, explicitly state uncertainty and propose next steps

Streaming Intermediate Steps

Stream operational progress updates such as:

- Planning
- Searching the web
- Fetching sources
- Selecting relevant context
- Generating answer with citations

Use Streamlit updates, CLI streaming prints, or FastAPI streaming responses. Do not stream hidden chain-of-thought.

Part 3: Evaluation Harness (Required)

Implement an evaluation setup that tests the agent on a **sample question-answer dataset**.

Expectations

- Create or curate a dataset of questions that meaningfully exercise the system (factual, multi-hop, comparison, insufficient-evidence, conflicting sources).
- Define your own evaluation approach and metrics that best measure:
 - grounding and citation quality
 - correctness and usefulness of answers
 - handling of uncertainty and conflicts
 - robustness across multiple turns and sessions
- Implement a runnable evaluation script that:
 - executes the agent on the dataset
 - records outputs, citations, and any intermediate artifacts you find useful
 - produces a clear summary of results

There is no prescribed metric set—part of the assignment is demonstrating good judgment in choosing *what* to measure and *why*.

Deliverables

- Working app (Streamlit, Gradio or your own UI)
- Web research ingestion via Tavily / Serper / Parallel
- Persistent sessions with conversation and turn history
- Citation-grounded answers with URL/title/domain references
- Streaming intermediate step updates
- Evaluation harness with clearly explained metrics and results
- README including:
 - Video Demo
 - setup and run instructions
 - design note (Part 1)
 - example conversations
 - evaluation methodology and findings
 - limitations and future improvements

Evaluation Criteria

- Soundness of chosen evaluation metrics and rationale
- Citation integrity and grounding to real web sources
- Quality of context selection and conflict handling
- Session and context management across the agent graph
- Code quality, modularity, and error handling

Submission

Files



Upload File(s)

or drag and drop here

Notes

Write here...

Submit Assignment

Powered by **Ashby**

[Privacy Policy](#)

[Security](#)

[Vulnerability Disclosure](#)